USER DOCUMENTATION/ MANUAL

Unlike code documents, user documents are usually far divorced from the source code of the program, and instead simply describe how it is used.

In the case of a software library the code documents and user documents could be effectively equivalent and are worth conjoining, but for a general application this is not often true. On the other hand, the lisp machine grew out of a tradition in which every piece of code had an attached documentation string. In combination with strong search capabilities (based on a Unix-like apropos command), and online sources, Lisp machine users could look up documentation and paste the associated function directly into their own code. This level of ease of use is unheard of in putatively more modern systems.

Typically, the user documentation describes each feature of the program, and the various steps required to invoke it. A good user document can also go so far as to provide thorough troubleshooting assistance. It is very important for user documents to not be confusing, and for them to be up to date. User documents need not be organized in any particular way, but it is very important for them to have a thorough index. Consistency and simplicity are also very valuable. User documentation is considered to constitute a contract specifying what the software will do and should be free from undocumented features.

There are three broad ways in which user documentation can be organized. A **tutorial approach** is considered the most useful for a new user, in which they are guided through each step of accomplishing particular tasks. A **thematic approach**, where chapters or sections concentrate on one particular area of interest, is of more general use to an intermediate user. The final type of

organizing principle is one in which commands or tasks are simply listed alphabetically, often via cross-referenced indices. This latter approach is of the greatest use to advanced users who know exactly what sort of information they are looking for. A common complaint among users regarding software documentation is that only one of these three approaches was taken to the near-exclusion of the other two

It is common to limit provided software documentation for personal computers to online help that give only reference information on commands or menu items. The job of tutoring new users or helping more experienced users get the most out of a program is left to private publishers, who are often given significant assistance by the software developer.

4.1 The following is a generic user manual structure:

1. Introduction
   - The product - introduce the product to the user.
   - The user manual
     - Scope/Purpose
     - Flow
     - Conventions
     - Glossary
2. Installing the software (assuming it's not a separate guide)
   - System requirements
     - Platform Support
   - Information/resources required in the process of installation
   - Installation steps

At this point there is usually a decision to be made about how to depict installation procedures for different platforms. The main criteriion here is how different the procedures are - if the steps are drastically different, you will have to

explain the procedure separately for each platform. But if the steps are not very different, you could choose the most common platform as your base, and wherever the steps are different, indicate the steps for the different platforms as indented text.

3. Using the software
    - Introduction
        - Purpose of the software
        - What it does and does not do (list the exact tasks)
        - User levels and the implications (segregate the user and admin level tasks)
    - Best configuration (for example, best viewed in 640x480 resolution)
    - Invoking the software
    - Interface elements
    - Steps to perform the required tasks
4. Administration
    - Reiterate the administration level tasks
    - Segregate (if possible) into administration, maintenance and troubleshooting functions, and then get into explanations
    - Always lead in to a task with scenarios (for example, you need to shut down the server over the weekends and at the end of the day - here's how...)
    - Also, try and bring out exceptional scenarios at the same time (to continue the above example, the administrator would not shut down the server over the weekend if there has been a request for remote access by one of the users)
5. Troubleshooting
    - For each error condition describe:
        - The error message displayed
        - What it means and what is the implication with respect to the attempted action (for example, the user will have to re-enter information)

- Steps to take to rectify the error

6. Appendix

An appendix allows you to expound on peripheral information that would be detracting when given in the main body. Detailed diagrams, flow charts, or references to books/tutorials on related software could be included here.

Here are some quick tips to assist you in developing this structure:

- Be visual: The most comforting thing for the user will be to see on screen what they've seen on the manual's pages, or vice-versa. Try and use screen grabs and small schematic diagrams wherever appropriate.
- Importance of relevant analogies: Essential if your software introduces concepts new to the user.
  - Use of transition words: "because", "therefore" and "consequently" are powerful words when talking about cause-effect relationships that the user isn't aware of.

4.2 Need to review

User reviews are a tad trickier than the others are because of the lack of resources. First, you may not have access to the actual users to review your document. And second, they may not really be motivated at that point to take the time to review your document. The workaround is to use your marketing and QA departments, and perhaps the people from the customer's end who are involved in the project.

Once the reviews are in, you need to get down to implementing the changes suggested. One tip would be to start revision on a document only after all the review comments are in. Also, while we won't get into the art of accepting feedback, you have to be in control of the changes that you agree to make. While you can change the information quite a bit at the time of the first review, you

should try and restrict your changes to corrections only after the second review; structural changes this late in the process will throw you off.

A characteristic of documentation is that if you notice even one inaccuracy in a document, it will put you off going through the rest of it. The gravity of this increases manifold when you're talking about a user who's looking to this document to understand your software. Ensuring that your manual reflects the latest version of the software is crucial, and this is where tying the document version number with that of the software comes in.

Another consideration here is version nomenclature. You could tie this in with the software, using x.y nomenclature that has x changing with every baseline change and y changing for every intermediate release of the document. Also, when you revise the document, you should record the reason/description of the change in the document's revision log.

With all of this behind you, you will finally be ready to release the manual. The following frills will complete the package:

1. Cover page
   - The name of the software should be written in accordance with the brand decided.
   - The version number of the software should be clearly stated.
   - The name of the developer with address and contact numbers.
2. Table of contents
   - The topics should be linked to the matter inside.
3. Notifications for proprietorship and confidentiality
4. Headers and footers
   - Headers could include the project name and version number of the document.
   - Footers can have the page numbers and a short confidentiality notice.

It might also be a good idea to include a feedback form as the last page, as your users will probably get back to you with suggestions. This will be especially useful if there is a second phase of development for the software.